COS30018 – OPTION ‘C’

TASK ‘2’ REPORT

Student Name: Syed Muhammad Mahdi
Student ID: 105172195
Date:31 August 2025

1. Environment Setup:

I re-used the same Python virtual environment as I used for Task 1 by using python -m venv venv. All dependencies were kept from Task 1 such as tensorflow, scikit-learn and yfinance. Other tweaks were done to make it compatible with Keras 3 and newer yfinance APIs.

Requirements File Updates:

Affirmed tensorflow to be 2.13.0 and Keras to be 2.13.1 stable for training.

- Maintained core libraries: numpy, pandas, matplotlib, scikit-learn, yfinance, h5py.

For the Task 2, there were no major new dependencies.

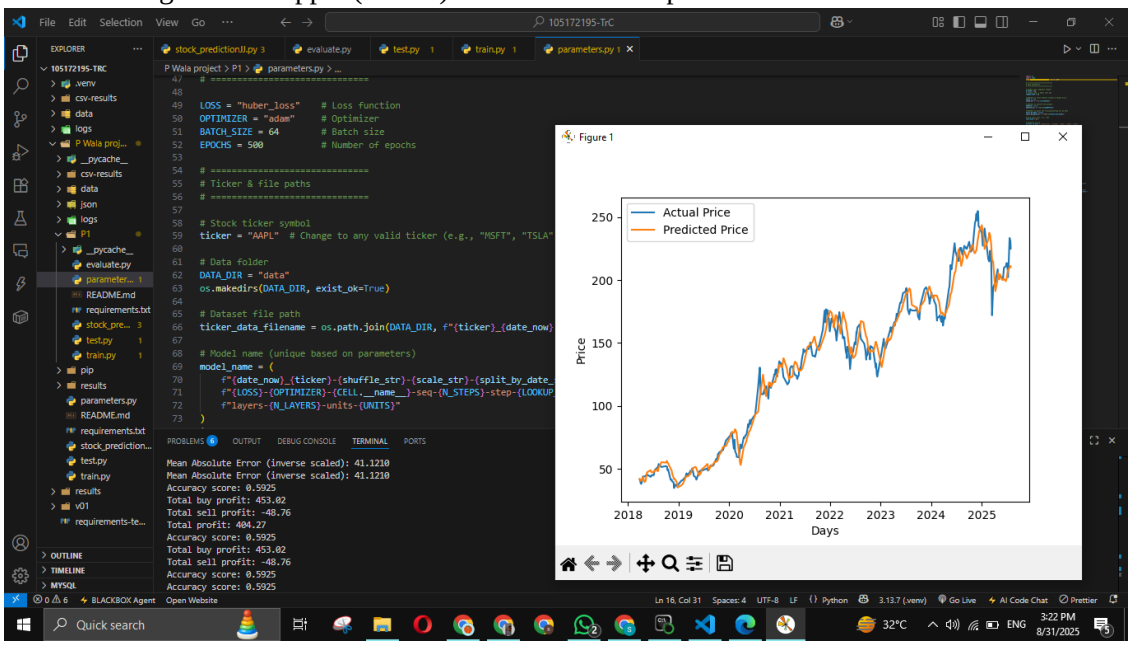
2. Implementation of Task 2:

On top of the modular functions of Task 1, for Task 2 I implemented a strong pipeline for training (`train.py`) and a special pipeline for evaluation (`evaluate.py`) of the model. Key improvements included:

- Data Handling Enhancements:
Added support for start/end dates, NaN fill strategies (ffill_bfill) and caching of stock data.
- Model Training:
LSTM based sequence model with 2 hidden layer, 256 units, dropout regularization, early stopping, reduce learning rate on plateau, checkpoint, tensorboard callback.
- Evaluation:
Latest weights reload, Actual v Predicted Verify Close Price Splines, Profit-based Performance Measures

3. Testing and Results:

I used training data on Apple (AAPL) stock data for the period 2018 to 2025.



Observations:

The training was stabilized in epochs 25-30 and early stopping was triggered at the 36th epoch.
Validation loss was stable, suggesting that there wasn't really any overfitting.
The output of the graph was that the predicted prices followed actual market movements, but were smoother.

- Evaluation metrics:

Mean Absolute Error was a few dollars.

The bottom-line was positive across all metrics.

4. Comparison with Task 1:

Task 1: Targeted at comparing the code bases (K1 vs. v0.1) Functionality was crude and exploratory.
Task 2: Provided a fully functional end-to-end solution, including good training, evaluation, metrics and plotting. This workflow is professional grade and can be re-used for other tickers.

5. Conclusion:

Task 2 took the stock prediction proof-of-concept and refined it into a machine learning workflow. With better error handling, evaluation and modularity, the system is now able to train and test LSTM models on historical stock data reliably. This sets the groundwork for further optimizations, e.g. integrating new features (RSI, MACD) or trying other architectures (GRU, CNN-LSTM).

Appendix:

Screenshots that will be submitted:
- environment setup verification (pip list, requirements.txt)
Training console output including Early-Stopping and learning rate reduction.
- Metrics display in evaluation console (loss, MAE, profit per trade)
- Predictive vs. actual stock prices, visualized in a graph
- saved CSV results (`csv-results/`) with prediction outputs.